Multimedia Signal processing

Prof. Marco Tagliasacchi

July, 4 2012

1. **Question (8 pts)**:
   Consider two sources, X and Y, that emit symbols according to the following joint p.m.f.

$$\Pr\{X = i, Y = j\} = w_{i,j}, \tag{1}$$

   where the probability values $w_{i,j}$ are defined in the matrix below (i: rows, j: columns)

$$\mathbf{W} = \frac{1}{8} \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{2}$$

   (a) Compute the entropy of both sources, i.e., $H(X)$ and $H(Y)$.

   (b) Compute the joint entropy $H(X, Y)$

   (c) Design an Huffman code for the source X and compare the average code length with $H(X)$.

   (d) Design an Huffman code to jointly encode X and Y and compare the average code length with $H(X, Y)$.

   Solution

   (a) The marginals are given by

$$\Pr\{X = 1\} = \frac{1}{4} \tag{3}$$

$$\Pr\{X = 2\} = \frac{1}{4} \tag{4}$$

$$\Pr\{X = 3\} = \frac{3}{8} \tag{5}$$

$$\Pr\{X = 4\} = \frac{1}{8} \tag{6}$$

$$\Pr\{Y = 1\} = \frac{1}{8} \tag{7}$$

$$\Pr\{Y = 2\} = \frac{1}{4} \tag{8}$$

$$\Pr\{Y = 3\} = \frac{1}{2} \tag{9}$$

$$\Pr\{Y = 4\} = \frac{1}{8} \tag{10}$$

The entropy values are equal to

$$H(X) = -\frac{1}{4}\log_2\frac{1}{4} - \frac{1}{4}\log_2\frac{1}{4} - \frac{3}{8}\log_2\frac{3}{8} - \frac{1}{8}\log_2\frac{1}{8} = 1.9056 \tag{11}$$

$$H(Y) = -\frac{1}{8}\log_2\frac{1}{8} - \frac{1}{4}\log_2\frac{1}{4} - \frac{1}{2}\log_2\frac{1}{2} - \frac{1}{8}\log_2\frac{1}{8} = 1.75 \tag{12}$$

(b) The joint entropy is

$$H(X,Y) = -4\frac{1}{8}\log_2\frac{1}{8} - 2\frac{1}{4}\log_2\frac{1}{4} = 2.50 \tag{13}$$

(c) A Huffman code for the source X is shown in Figure 1 and detailed in the table below.

symbol	prob.	codeword	length
1	1/4	111	3
2	1/4	10	2
3	3/8	0	1
4	1/8	110	3

The average code length is

$$L = 3\cdot(1/4) + 2\cdot(1/4) + 1\cdot(3/8) + 3\cdot(1/8) = 2 \text{ bits / symbol} \tag{14}$$

Hence $L_X > H(X)$ (with strict inequality, since the source probabilities are not dyadic). Note that in this case $L_X = \log_2 4 = 2$, i.e., no compression is achieved.

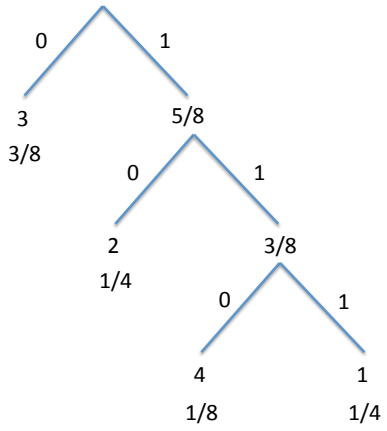


Figure 1: Huffman tree for source X.

(d) A Huffman code for jointly encoding X and Y is shown in Figure 2 and detailed in the table below.

symbol	prob.	codeword	length
(1,1)	1/8	101	3
(1,3)	1/8	100	3
(2,3)	1/4	01	2
(3,2)	1/4	00	2
(3,3)	1/8	111	3
(4,4)	1/8	110	3

The average code length is

$$L_{X,Y} = 3 \cdot (1/8) + 3 \cdot (1/8) + 2 \cdot (1/4) + 2 \cdot (1/4) + 3 \cdot (1/8) + 3 \cdot (1/8) = 2.5 \text{ bits/supersymbol} \quad (15)$$

Hence $L_{X,Y} = H(X,Y)$ (with strict equality, since the source probabilities are dyadic).
The number of bits per symbol is $L_{X,Y}/2 = 1.25$ bits/symbol.

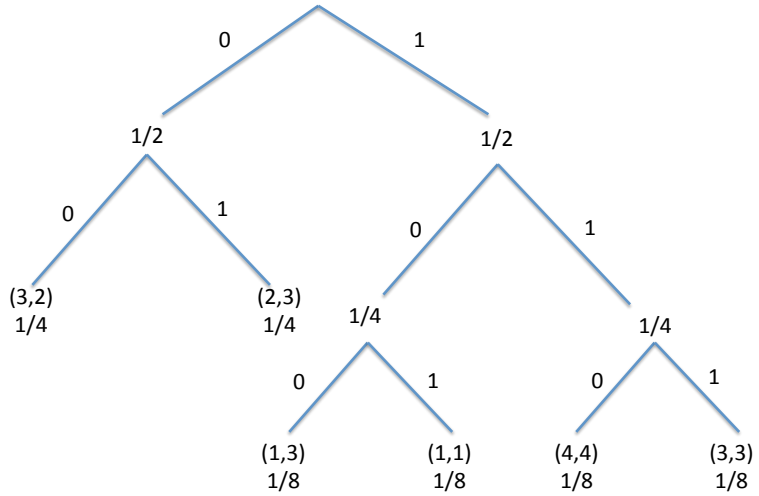


Figure 2: Huffman tree for the joint source X and Y.

2. **Question (5 pts)**
   (a) Illustrate the block diagram of an open loop vocoder.
   (b) Discuss the use of short and long term predictors.

   Solution

   See slide set `MMSP_2.1_Speech coding`.
3. **Question (5 pts)**
   (a) Illustrate the design of the compander-expander quantizer.

(b) Discuss the coding efficiency with respect to the uniform quantizer and the Lloyd-Max quantizer.

Solution

See slide set `MMSP_1.2_Lossy_Coding`.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**
   (a) Consider a speech coding system
   - [T] [F] The number of prediction coefficients used in the short term predictor is approximately equal to 100.
     FALSE. The number of coefficients is about 6-20. This number is generally enough to capture the formant structure of the speech spectrum.
   - [T] [F] Direct quantization of prediction coefficients might lead to instability at the decoder
     TRUE. Direct quantization might cause the poles of the shaping filter to be outside the unit circle, thus causing instability at the decoder side.
   - [T] [F] The number of zeros of the whitening filter $A(z)$ depends on the number of formants below 400 Hz
     FALSE. The number of zeros depends on the number of formants below 4 kHz.

   (b) Consider an image coding system
   - [T] [F] The 2D transform is invertible.
     TRUE. The transform is invertible up to the finite arithmetic precision. Integer transforms are perfectly invertible.
   - [T] [F] The coding efficiency increases when the block size is increased.
     TRUE. Increasing the block size it is possible to capture inter-pixel correlations at longer distance (at the cost of higher computational complexity)
   - [T] [F] The complexity of the 2D DCT is $O(N^2 \log N)$
     TRUE. The 2D DCT is separable. N DCT transforms (at cost $O(N \log N)$) are computed along the rows and columns.
   - [T] [F] The color components in natural images (e.g., RGB components) are statistically independent.
     FALSE. The color components are dependent, due to the overlapping between the spectral response of the acquisition sensors. Hence, a colorspace transformation is often employed to separate the luma from the chroma components.

   (c) Consider a video coding system.
   - [T] [F] The average MSE distortion of prediction residuals increases when the block size increases.
     TRUE. Larger blocks induce larger errors in the motion compensation phase.
   - [T] [F] The average MSE distortion of prediction residuals of full search motion estimation is smaller than (or equal to) that of fast search.
     TRUE. Fast search avoids to visit all motion vector candidates.
   - [T] [F] Motion vectors of neighboring blocks are statistically dependent.
     TRUE. This is exploited, e.g., in fast search (to initialize a good candidate), or in entropy coding of motion vectors.

5. **Question (Matlab - 10pts)**
   Local image features are routinely used in several computer vision applications, including augmented reality (e.g Google Goggles). Their success is due to the ability of summarizing the input image in a way that is invariant to several transformations (scale, rotation, luminance, viewpoint etc.).
   You are requested to analyze the performance of lossless and lossy compression when applied to the feature set.
   (a) Load the file `feature_set.mat`. It contains a collection of 64-dimensional SURF local features extracted from a test image (each feature corresponds to one column in the feature set).

(b) Lossless coding:

- Assume the feature set as a *single* discrete memoryless source X, which emits symbols quantized with 8 bits. Compute the empirical entropy $H(X)$ (in bits / symbol) and the expected rate R (in bits / vector) for lossless coding a single feature vector.
- Now assume that each of the 64 dimensions of the feature set as a different source, i.e., X_i ($i = 1 \dots 64$). Compute the empirical entropies $H(X_i)$ and the expected rate $R = \sum_{i=1}^{64} H(X_i)$.

(c) Lossy coding: Encode the feature set with a PCM coding scheme. Use a uniform quantizer with granular region between -0.5 and 0.5 and with $M = \{2, 4, 8, 16, 32, 64, 128\}$ levels. Plot the rate-distortion curve.

Solution

```
clear all
close all
clc
load feature_set;

[rows cols] = size(feature_set);
% rows = feature dimension (64)
% cols = number of features in the set

%Assume the feature set as a discrete memoryless source X, which emits
%symbols quantized with 8 bits. Compute the empirical entropy H(X) and
%the expected rate R for lossless coding a single feature vector.

h = hist(feature_set(:),2^8);
p = h/sum(h);
p = p(p~=0);
H = -sum(p.*log2(p));
R = rows*H;
display(['Expected rate: ' num2str(R) ' bits per feature vector']);

%Now assume each of the i-th dimension of the feature set as a different source Xi (i = 1 . . . 64).
%Compute the empirical entropies H(Xi) and the expected rate R = sum(H(Xi)).

Hi = zeros(1,rows);
for i = 1:rows
    h = hist(feature_set(i,:),2^8);
    p = h/sum(h);
    p = p(p~=0);
    Hi(i) = -sum(p.*log2(p));
end
R = sum(Hi);
display(['Expected rate: ' num2str(R) ' bits per feature vector']);

%Encode the feature set with a PCM coding scheme. Use a uniform quantizer with granular region
%between -0.5 and 0.5 and with M = {2, 4, 8, 16, 32, 64, 128} levels. Plot the rate-distortion curve

M = [2 5 8 16 32 64 128];
lb = -0.5;
ub = 0.5;

SNR = zeros(1,length(M));
im = 1;
for m=M
   step = (ub-lb)/m;
   feature_q = step.*round(feature_set./step);
   SNR(im) = 10*log10(var(feature_set(:))/var(feature_set(:) - feature_q(:)));
   im = im+1;
end

figure(1)
plot(log2(M)*rows,SNR);
```